

Deep Generative Models

Chapter 4: Generative Adversarial Networks

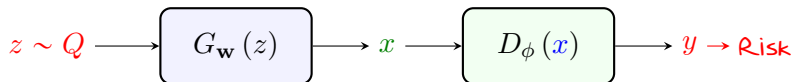
Ali Bereyhi

`ali.bereyhi@utoronto.ca`

Department of Electrical and Computer Engineering
University of Toronto

Summer 2026

WGAN: Alternative Paradigm



To train this architecture, we can follow two paradigms

- ✓ **Vanilla** GAN solves **the min-max game**
 - ↳ It **implicitly** maximizes the likelihood $\equiv$ minimizes the JS divergence
- ➔ **Wasserstein** GAN which is less intuitive
 - ↳ It **implicitly** minimizes the so-called Wasserstein distance

Preliminaries: Coupling Set

To understand WGAN paradigm: we need to define *Wasserstein Distance*

Coupling Set

Let P and $\hat{P}$ be two distributions defined on data space $\mathbb{X}$: the *coupling* set of P and $\hat{P}$ is defined as

$$\Pi_{P,\hat{P}} = \left\{ Q(x, \hat{x}) : \sum_{\hat{x}} Q(x, \hat{x}) = P(x) \text{ and } \sum_x Q(x, \hat{x}) = \hat{P}(\hat{x}) \right\}$$

Wasserstein Distance

Wasserstein distance between P and $\hat{P}$ is defined as

$$D_W(P \parallel \hat{P}) = \min_{Q \in \Pi_{P,\hat{P}}} \mathbb{E}_{(x,\hat{x}) \sim Q} \{ \|x - \hat{x}\| \}$$

Wasserstein vs JS: Example

Say we have *latent variable* z uniformly distributed on $(0, 1)$, i.e.,¹

$$z \sim \text{Unif}(0, 1)$$

Data x is related to z as $x = [0, z]$

x lies on a 1D manifold in the 2D space!

We have the following generator

For some learnable $\theta \in [-1, 1]$, the generator generates $\hat{x}$ from z as

$$\hat{x} = G_{\theta}(z) = [\theta, z]$$

Let's see how various divergence metrics between x and $\hat{x}$ look like

¹This example is taken from [Wasserstein GAN](#) paper by [M. Arjovsky et al.](#)

Wasserstein vs JS: Example

Since $\hat{x} = [\theta, z]$ for $\theta \in [-1, 1]$ and *latent* $z \in [0, 1]$, we have

$$\mathbb{X} = [-1, 1] \times [0, 1]$$

We can also present the *data* and *model* distributions as

- Data* is *uniformly* distributed on the *line* $x_1 = 0$

$$P(x) = \boxed{\delta(x_1)}$$

dirac impulse at $x_1 = 0$

- Model* output is *uniformly* distributed on the *line* $x_1 = \theta$

$$\hat{P}_\theta(x) = \boxed{\delta(x_1 - \theta)}$$

dirac impulse at $x_1 = \theta$

Wasserstein vs JS: Example

For JS divergence, we have

$$D_{\text{JS}} \left(P \| \hat{P}_\theta \right) = D_{\text{KL}} \left(P \| A_{P, \hat{P}_\theta} \right) + D_{\text{KL}} \left(\hat{P} \| A_{P, \hat{P}_\theta} \right)$$

The average distribution in this case reads

$$A_{P, \hat{P}_\theta} = \frac{\delta(x_1) + \delta(x_1 - \theta)}{2}$$

If $\theta = 0$; then, we have

$$\begin{aligned} D_{\text{KL}} \left(P \| A_{P, \hat{P}_\theta} \right) &= \int_0^1 \int_{-1}^1 \delta(x_1) \log \frac{2\delta(x_1)}{\delta(x_1) + \delta(x_1)} dx_1 dx_2 \\ &= \int_0^1 \log 1 dx_2 = 0 \end{aligned}$$

Wasserstein vs JS: Example

If $\theta \neq 0$; then, we have

$$\begin{aligned}
 D_{\text{KL}} \left(P \parallel A_{P, \hat{P}_\theta} \right) &= \int_0^1 \int_{-1}^1 \delta(x_1) \log \frac{2\delta(x_1)}{\delta(x_1) + \underbrace{\delta(x_1 - \theta)}_{0 \text{ for } [0^-, 0^+]}} dx_1 dx_2 \\
 &= \log 2
 \end{aligned}$$

We can show that $D_{\text{KL}} \left(\hat{P}_\theta \parallel A_{P, \hat{P}} \right)$ reads the same, i.e.,

$$D_{\text{KL}} \left(\hat{P}_\theta \parallel A_{P, \hat{P}_\theta} \right) = \begin{cases} \log 2 & \theta \neq 0 \\ 0 & \theta = 0 \end{cases}$$

Wasserstein vs JS: Example

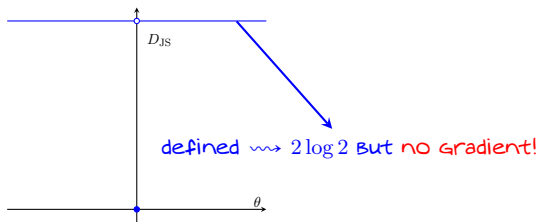
So, JS divergence reads

$$D_{\text{JS}} \left(P \parallel \hat{P}_{\theta} \right) = \begin{cases} 2 \log 2 & \theta \neq 0 \\ 0 & \theta = 0 \end{cases}$$

which is *minimized* when

data distribution $P \equiv$ model distribution $\hat{P}_{\theta}$

as we expected!



Wasserstein vs JS: Example

For the Wasserstein distance, we have

$$D_W \left(P \parallel \hat{P}_\theta \right) = \min_{Q \in \Pi_{P, \hat{P}}} \mathbb{E}_{(x, \hat{x}) \sim Q} \{ \|x - \hat{x}\| \}$$

Let us compute the expectation of an *arbitrary* Q

$$\mathbb{E}_{(x, \hat{x}) \sim Q} \{ \| [0, z] - [\theta, z] \| \} = \mathbb{E}_{(x, \hat{x}) \sim Q} \{ |\theta| \} = |\theta|$$

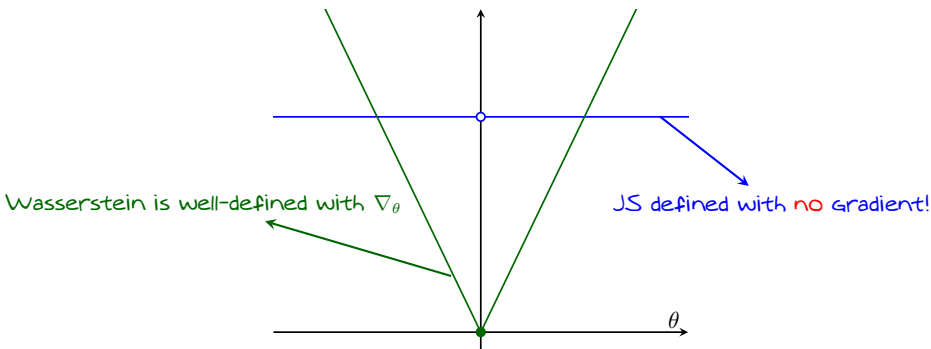
This is independent of Q ; thus, we could say

$$D_W \left(P \parallel \hat{P}_\theta \right) = |\theta|$$

which is again *minimized* when $P = \hat{P}_\theta$!

Wasserstein vs JS: Example

All metrics are *minimized* when $P = \hat{P}_\theta$ but look differently against θ



Moral of Story

It might be better to *minimize the Wasserstein distance* between the data and generator distribution instead of *JS*

Minimizing Wasserstein Distance

We concluded that GAN *implicitly* learns

$$\hat{P}^{\star} = \operatorname{argmin}_{\hat{P}} D_{\text{JS}} \left(P \parallel \hat{P} \right)$$

by training generator G . Let us now replace this by *Wasserstein distance*

$$\begin{aligned} \hat{P}^{\star} &= \operatorname{argmin}_{\hat{P}} D_{\text{W}} \left(P \parallel \hat{P} \right) \\ &= \operatorname{argmin}_{\hat{P}} \min_{Q \in \Pi_{P, \hat{P}}} \mathbb{E}_{(x, \hat{x}) \sim Q} \{ \|x - \hat{x}\| \} \end{aligned}$$

This does *not* seem tractable!

- ↳ It does not sound feasible to *search* $\Pi_{P, \hat{P}}$
- ↳ *Kantorovich-Rubinstein duality* lets us do this *indirectly*

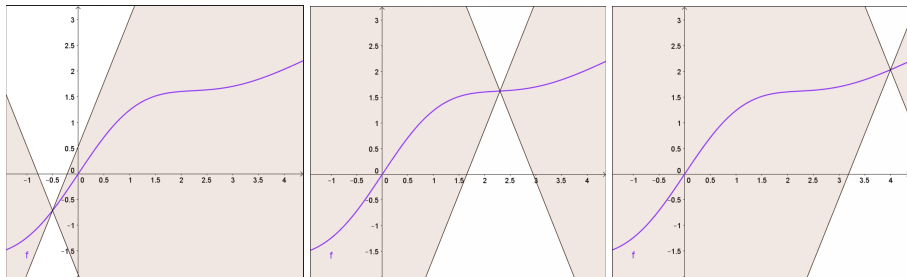
Lipschitz Continuity

L -Lipschitz Function

$f : \mathbb{X} \mapsto \mathbb{R}^m$ is Lipschitz if for any x and $\hat{x} \in \mathbb{X}$

$$\|f(x) - f(\hat{x})\| \leq L \|x - \hat{x}\|$$

with L being the Lipschitz constant



Lipschitz Function Space

L -Lipschitz Function Space

The set of all functions $f : \mathbb{X} \mapsto \mathbb{R}^m$ that are L -Lipschitz

$$\mathbb{L}_L = \{f : f \text{ is } L\text{-Lipschitz}\}$$

! Attention

In practice, we work with *computational* models of form

$$f_{\mathbf{w}} : \mathbb{X} \mapsto \mathbb{R}^m$$

for such models, we have a set of parameters $\mathbb{W}_L$ such that

$$\text{for all } \mathbf{w} \in \mathbb{W}_L \rightsquigarrow f_{\mathbf{w}} \in \mathbb{L}_L$$

Minimizing Wasserstein Distance

Kantorovich-Rubinstein Duality

Wasserstein distance between P and $\hat{P}$ is computed as

$$D_W \left(P \parallel \hat{P} \right) = \max_{D \in \mathbb{L}_1} \mathbb{E}_{x \sim P} \{ D(x) \} - \mathbb{E}_{x \sim \hat{P}} \{ D(x) \}$$

Kantorovich-Rubinstein Duality: Computational Modeling

Wasserstein distance between P and $\hat{P}$ is estimated by model D_ϕ as

$$\hat{D}_W \left(P \parallel \hat{P} \right) = \max_{\phi \in \Phi_1} \hat{\mathbb{E}}_{x \sim P} \{ D_\phi(x) \} - \hat{\mathbb{E}}_{x \sim \hat{P}} \{ D_\phi(x) \}$$

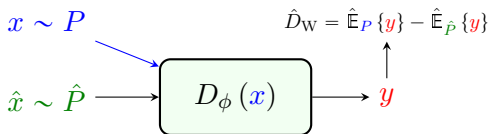
with Φ_1 is the set of parameters with which D_ϕ is *1-Lipschitz*

Minimizing Wasserstein Distance

Back to our problem: we want to *minimize Wasserstein distance*

$$\begin{aligned}\hat{P}^* &= \underset{\hat{P}}{\operatorname{argmin}} D_W \left(P \parallel \hat{P} \right) \\ &= \underset{\hat{P}}{\operatorname{argmin}} \max_{D \in \mathbb{L}_1} \mathbb{E}_{x \sim P} \{ D(x) \} - \mathbb{E}_{x \sim \hat{P}} \{ D(x) \}\end{aligned}$$

We can estimate it by learning a *computational discriminator* D_ϕ



as long as we *search over* Φ_1 to make sure it's *1-Lipschitz*

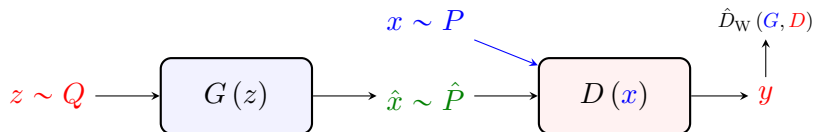
Minimizing Wasserstein Distance

- + But, we need to do this *implicitly*! Right?!
- Exactly! We can use a *generator*

Sample $z \sim Q(z)$ and let $\hat{x} = G(z) \sim \hat{P}$: we find $\hat{P}^*$ *implicitly* as

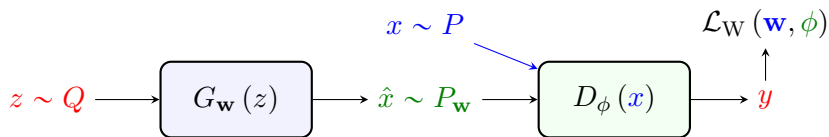
$$G^* = \underset{G}{\operatorname{argmin}} \max_{D \in \mathbb{L}_1} \mathbb{E}_{x \sim P} \{D(x)\} - \mathbb{E}_{z \sim Q} \{D(G(z))\}$$

Bingo! This is a GAN with *alternative min-max objective*



This is what we call *Wasserstein GAN*

Wasserstein GAN: Alternative Paradigm to Train GAN



Wasserstein GAN (Computational)

Consider a GAN with **generator** $G_{\mathbf{w}}$ and **discriminator** D_{ϕ} which is 1-Lipschitz for $\phi \in \Phi_1$: Wasserstein GAN learns through the **min-max game**

$$\min_{\mathbf{w}} \max_{\phi \in \Phi_1} \mathcal{L}_{\mathbf{W}}(\mathbf{w}, \phi)$$

with objective function

$$\mathcal{L}_{\mathbf{W}}(\mathbf{w}, \phi) = \hat{\mathbb{E}}_{x \sim P} \{D_{\phi}(x)\} - \hat{\mathbb{E}}_{z \sim Q} \{D_{\phi}(G_{\mathbf{w}}(z))\}$$

W-GAN: Training Loop

Train_WGAN($\mathbb{D}$:dataset):

```

1: Initiate the generator  $G_{\mathbf{w}}$  and discriminator  $D_{\phi}$  with some  $\mathbf{w}$  and  $\phi$ 
2: for multiple epochs do
3:   Sample a batch of data samples  $\{x^j : j = 1, \dots, n\}$  from  $\mathbb{D}$ 
4:   Sample latents  $\{z^j : j = 1, \dots, n\}$  using  $Q$  and compute  $\hat{x}^j \leftarrow G_{\mathbf{w}}(z^j)$ 
5:   for  $\xi = 1, \dots, \Xi$  do
6:     for  $j = 1, \dots, n$  do
7:       Compute  $\mathcal{L}^j = D_{\phi}(x^j) - D_{\phi}(\hat{x}^j)$ 
8:       Backpropagate over discriminator to compute  $\nabla_{\phi} \mathcal{L}^j$ 
9:       if  $\xi = \Xi$  then Backpropagate over generator to compute  $\nabla_{\mathbf{w}} \mathcal{L}^j$ 
10:    end for
11:    Update  $\phi$  using  $\text{Opt\_avg} \{-\nabla_{\phi} \mathcal{L}^j\}$  and make sure  $D_{\phi}$  is 1-Lipschitz
12:  end for
13:  Update  $\mathbf{w}$  using  $\text{Opt\_avg} \{\nabla_{\mathbf{w}} \mathcal{L}^j\}$ 
14: end for
15: return trained generator  $G_{\mathbf{w}}$ 

```

Wasserstein vs Vanilla GAN

Looking at **Wasserstein GAN** objective

$$\min_{\mathbf{w}} \max_{\phi \in \Phi_1} \hat{\mathbb{E}}_{x \sim P} \{D_{\phi}(x)\} - \hat{\mathbb{E}}_{z \sim Q} \{D_{\phi}(G_{\mathbf{w}}(z))\}$$

and comparing it with **vanilla GAN**

$$\min_{\mathbf{w}} \max_{\phi} \hat{\mathbb{E}}_{x \sim P} \{\log D_{\phi}(x)\} + \hat{\mathbb{E}}_{z \sim Q} \{\log (1 - D_{\phi}(G_{\mathbf{w}}(z)))\}$$

we can say

- ① Objective does **not** use **cross-entropy** anymore, i.e., **no log**
- ② D_{ϕ} in WGAN does **not** need to be **Sigmoid output** anymore
- ③ D_{ϕ} in WGAN should be **kept 1-Lipschitz**

Imposing Lipschitz Continuity

There are various approaches to restrict *discriminator* being *Lipschitz*

- ① *Weight clipping* which is the simplest approach
 - ↳ We clip the weights periodically throughout training
- ② *Gradient penalty* which is more *sophisticated*
 - ↳ We add $(\|\nabla_x D(x)\| - 1)^2$ as penalty
 - ↳ This assures that output variation does not get extremely large
- ③ *Spectral normalization* which is again an *advanced approach*
 - ↳ We normalize each layer by its *spectral norm*

$$\mathbf{W}_\ell \leftarrow \frac{\mathbf{W}_\ell}{\max \{\text{eig}(\mathbf{W}_\ell)\}}$$